

PRODUCTION AI ENGINEERING · T1 · EVIDENCE REAL VS SIMULATED

T1 Real vs Simulated

Which parts of the Agent Identity POC are real code on the claim's path, which are simulated systems of record, and what each choice means for the evidence.

T1

T1 · Production AI Engineering

3 min read · Published 2026-10-03 · Real vs simulated

The rule

IMPLEMENTED

SIMULATED

`agent_identity_poc/runs/2026-10-03-recorded/manifest.json` lists the same split

Everything a claim depends on is real code, exercised by the run. Everything that only produces or records inputs (the enterprise systems, the identity provider, the clock) is simulated, deterministic, and shaped like its real counterpart. The agents follow fixed plans on purpose: T1's claims are about who acts and with what authority, and they have to hold whatever a model proposes.

Component by component

Component	Status	Where	What it means for the evidence
Ingress authentication of each head (webhook, console, workflow, CI)	Real	<code>aid/directory.py</code> , <code>aid/platform.py</code>	Who invoked is established by code, not asserted
Token exchange, delegation, scope intersection, narrowing, depth limit, declared edges	Real	<code>aid/trust.py</code>	I1, I2 and I5 test the actual narrowing and actor-chain logic
Workload attestation check at the gateway (<code>cnf</code> against the presenter's SPIFFE id)	Real	<code>aid/trust.py</code> , <code>aid/attestation.py</code>	I4's replay rejection is the real check
SVID issuance and the attestation authority	Simulated	<code>aid/attestation.py</code>	No SPIRE; identities are issued from configuration
Capability gateway: registration, scope check, approval requirement	Real	<code>aid/gateway.py</code>	I2's DENY and every executed call go through it
Approvals bound to one call digest	Real	<code>aid/approvals.py</code>	Approval grants <code>deploy:rollback</code> once, for one call
Token broker: per capability class, audience-bound, 10-minute credentials	Real	<code>aid/broker.py</code> , <code>config/tool_identities.yaml</code>	I4's audience and expiry results come from it
Hash-chained platform audit with revocation handles	Real	<code>aid/audit.py</code>	I1's nine answers and the tamper test read it. Tamper-evident application audit, not an independently anchored ledger
Replay checks: execution token bound to the attested workload; tool credential bound to one audience and an expiry	Real	<code>aid/trust.py</code> , <code>aid/broker.py</code> , <code>aid/tools.py</code>	I4, including the approved rollback replayed from another workload (G03)
Evidence assertions: the 41 declared checks, the freeze and the byte-for-byte replay	Real	<code>aid/experiments.py</code> , <code>aid/freeze.py</code> , <code>tools/verify_run.py</code>	The run refuses to start if a declared criterion changed
Revocation levers (delegation, invoker credential, agent, workload, tool identity, shared account)	Real	<code>aid/directory.py</code> , <code>aid/platform.py</code>	I3 pulls each lever against running executions
The three identity models	Real	<code>aid/platform.py</code> (chain), <code>aid/baselines.py</code> (shared account, user token)	The shared account and user token are implemented, not described
Datadog (the monitor event)	Simulated	<code>aid/experiments.py</code> (<code>dd-evt-771204</code>)	Provenance is recorded as data; the integration is not exercised

Component	Status	Where	What it means for the evidence
Directory and enterprise identity provider	Simulated	<code>config/principals.yaml</code>	Principals, roles and delegations come from configuration; OIDC subjects are <code>(iss, sub)</code> pairs in the fixture, not a live IdP
Commercial authorization platform	Not used	<code>config/policies.yaml</code>	The gateway applies a minimal rule list; authorization is the next note
Enterprise approval system	Simulated	<code>aid/approvals.py</code> called by the experiments	The approver's decision is scripted; the binding to one call digest is real
Kubernetes, Jira, Slack, telemetry, and their own logs	Simulated	<code>aid/tools.py</code>	Each keeps its own log of exactly what it was shown; the Kubernetes entry mirrors the audit-event fields that matter here
Clock and human waiting	Simulated	<code>aid/config.py</code>	Pauses and expiry are exact and repeatable; the 37-minute wait costs nothing
Agents	Fixed plans	<code>aid/agents.py</code>	No model; the same calls in every run and every mode

Not implemented

Discussed in the technical edition, not exercised by the POC: a real identity provider and the RFC 8693 wire format, SPIRE attestation, DPoP or mTLS-bound tool credentials, Kubernetes impersonation headers, transaction tokens across services, token introspection for immediate revocation of self-contained tokens, and the authorization and policy engine the next note is about.

What would change with real systems

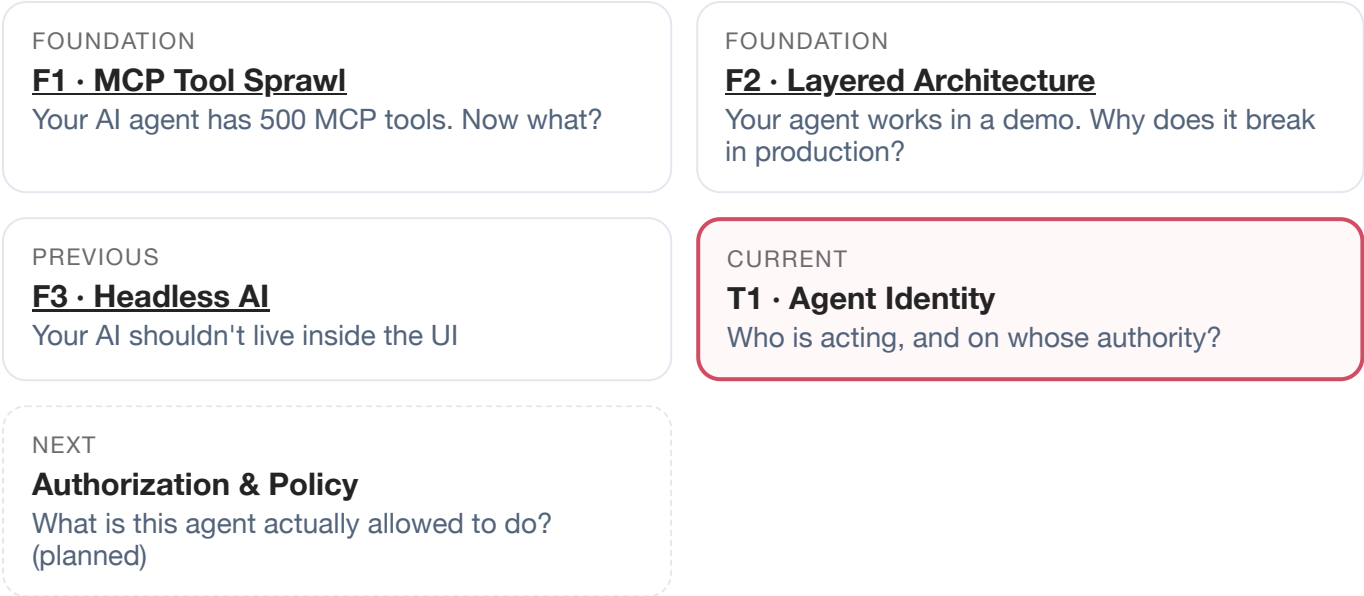
- **Latency and availability.** The broker and the exchange sit on the request path. The POC does not measure either; a production design needs caching of execution identities and a failure mode for the broker (fail closed for writes).
- **Revocation.** Real deployments can shorten the qualified window in the Evidence Check with introspection or sender-constrained tool credentials, where the tool supports them.
- **Logs.** Real tool logs carry more fields than the simulated ones, but not the chain: Kubernetes records the authenticated principal and the request, not the invoker, the agent or the approval behind it.

Evidence

Method

Agent identity

PRODUCTION AI ENGINEERING · THE SERIES SO FAR



[The full learning map \(Start Here\)](#)

T1 · Production AI Engineering

Production AI Engineering · T1 · Evidence. Every measured number here is substituted at build time from agent_identity_poc/runs/2026-10-03-recorded/facts.json; the enterprise systems are simulated and the run is deterministic. Figures carry their provenance.

- [Medium edition](#)
- [Technical deep dive](#)
- [Evidence Check](#)
- [POC README](#)
- [Run summary](#)
- [QA report](#)